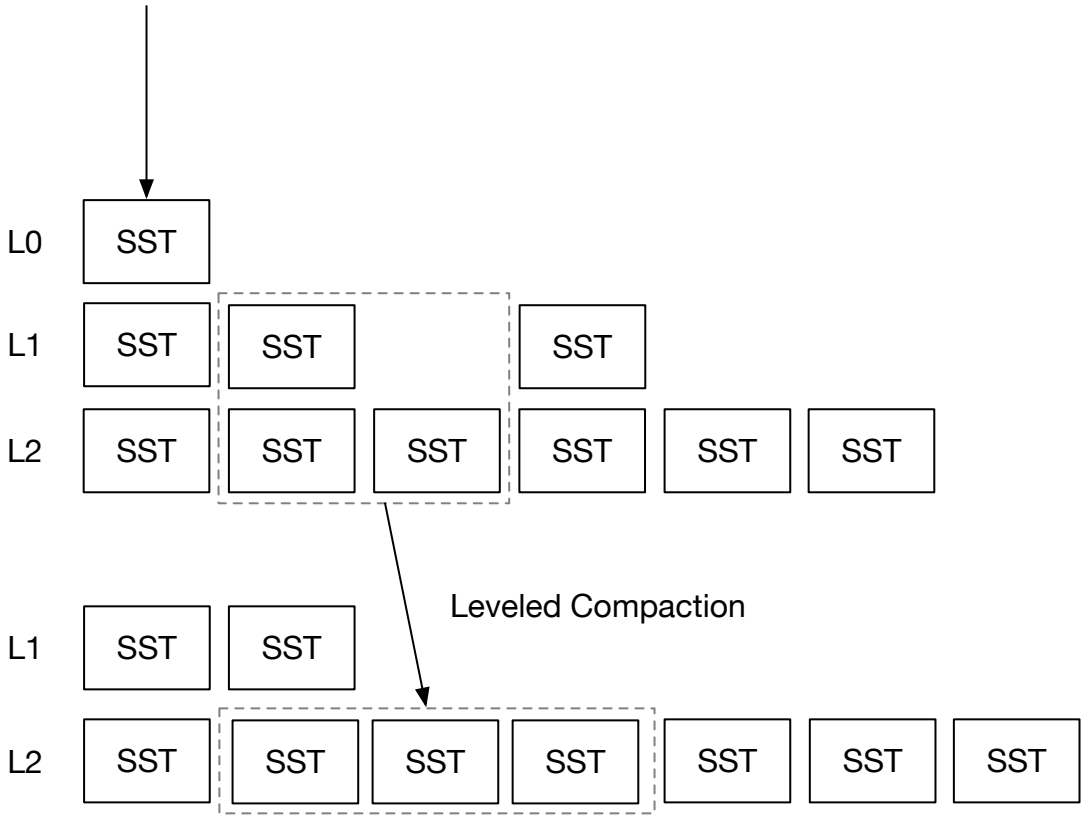
Leveled Compaction Strategy



In this chapter, you will:

- Implement a leveled compaction strategy and simulate it on the compaction simulator.
- Incorporate leveled compaction strategy into the system.

To copy the test cases into the starter code and run them,

```
cargo x copy-test --week 2 --day 4
cargo x scheck
```

> It might be helpful to take a look at week 2 overview before reading this chapter to have a general overview of compactions.

Task 1: Leveled Compaction

In chapter 2 day 2, you have implemented the simple leveled compaction strategies. However, the implementation has a few problems:

- Compaction always include a full level. Note that you cannot remove the old files until you finish the compaction, and therefore, your storage engine might use 2x storage space while the compaction is going on (if it is a full compaction). Tiered compaction

has the same problem. In this chapter, we will implement partial compaction that we select one SST from the upper level for compaction, instead of the full level.
- SSTs may be compacted across empty levels. As you have seen in the compaction simulator, when the LSM state is empty, and the engine flushes some L0 SSTs, these SSTs will be first compacted to L1, then from L1 to L2, etc. An optimal strategy is to directly place the SST from L0 to the lowest level possible, so as to avoid unnecessary write amplification.

In this chapter, you will implement a production-ready leveled compaction strategy. The strategy is the same as RocksDB's leveled compaction. You will need to modify:

```
src/compact/leveled.rs
```

To run the compaction simulator,

```
cargo run --bin compaction-simulator leveled
```

Task 1.1: Compute Target Sizes

In this compaction strategy, you will need to know the first/last key of each SST and the size of the SSTs. The compaction simulator will set up some mock SSTs for you to access.

You will need to compute the target sizes of the levels. Assume `base_level_size_mb` is 200MB and the number of levels (except L0) is 6. When the LSM state is empty, the target sizes will be:

```
[0 0 0 0 0 200MB]
```

Before the bottom level exceeds `base_level_size_mb`, all other intermediate levels will have target sizes of 0. The idea is that when the total amount of data is small, it's wasteful to create intermediate levels.

When the bottom level reaches or exceeds `base_level_size_mb`, we will compute the target size of the other levels by dividing the `level_size_multiplier` from the size. Assume the bottom level contains 300MB of data, and `level_size_multiplier=10`.

```
0 0 0 0 30MB 300MB
```

In addition, at most *one* level can have a positive target size below `base_level_size_mb`. Assume we now have 30GB files in the last level, the target sizes will be,

```
0 0 30MB 300MB 3GB 30GB
```

Notice in this case L1 and L2 have target size of 0, and L3 is the only level with a positive target size below `base_level_size_mb`.

Task 1.2: Decide Base Level

Now, let us solve the problem that SSTs may be compacted across empty levels in the simple leveled compaction strategy. When we compact L0 SSTs with lower levels, we do not directly put it to L1. Instead, we compact it with the first level with `target size > 0`. For example, when the target level sizes are:

```
0 0 0 0 30MB 300MB
```

We will compact L0 SSTs with L5 SSTs if the number of L0 SSTs reaches the `level0_file_num_compaction_trigger` threshold.

Now, you can generate L0 compaction tasks and run the compaction simulator.

```
--- After Flush ---
L0 (1): [23]
L1 (0): []
L2 (0): []
L3 (2): [19, 20]
L4 (6): [11, 12, 7, 8, 9, 10]

...

--- After Flush ---
L0 (2): [102, 103]
L1 (0): []
L2 (0): []
L3 (18): [42, 65, 86, 87, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 61, 62, 52,
34]
L4 (6): [11, 12, 7, 8, 9, 10]
```

The number of levels in the compaction simulator is 4. Therefore, the SSTs should be directly flushed to L3/L4.

Task 1.3: Decide Level Priorities

Now that we will need to handle compactions below L0. L0 compaction always has the top priority, thus you should compact L0 with other levels first if it reaches the threshold. After that, we can compute the compaction priorities of each level by `current_size / target_size`. We only compact levels with this ratio `> 1.0` The one with the largest ratio will be chosen for compaction with the lower level. For example, if we have:

```
L3: 200MB, target_size=20MB
L4: 202MB, target_size=200MB
L5: 1.9GB, target_size=2GB
L6: 20GB, target_size=20GB
```

The priority of compaction will be:

```
L3: 200MB/20MB = 10.0
L4: 202MB/200MB = 1.01
L5: 1.9GB/2GB = 0.95
```

L3 and L4 needs to be compacted with their lower level respectively, while L5 does not. And L3 has a larger ratio, and therefore we will produce a compaction task of L3 and L4. After the compaction is done, it is likely that we will schedule compactions of L4 and L5.

Task 1.4: Select SST to Compact

Now, let us solve the problem that compaction always include a full level from the simple leveled compaction strategy. When we decide to compact two levels, we always select the oldest SST from the upper level. You can know the time that the SST is produced by comparing the SST id.

There are other ways of choosing the compacting SST, for example, by looking into the number of delete tombstones. You can implement this as part of the bonus task.

After you choose the upper level SST, you will need to find all SSTs in the lower level with overlapping keys of the upper level SST. Then, you can generate a compaction task that contain exactly one SST in the upper level and overlapping SSTs in the lower level.

When the compaction completes, you will need to remove the SSTs from the state and insert new SSTs into the correct place. Note that you should keep SST ids ordered by first keys in all levels except L0.

Running the compaction simulator, you should see:

```
--- After Compaction ---
L0 (0): []
L1 (4): [222, 223, 208, 209]
L2 (5): [206, 196, 207, 212, 165]
L3 (11): [166, 120, 143, 144, 179, 148, 167, 140, 189, 180, 190]
L4 (22): [113, 85, 86, 36, 46, 37, 146, 100, 147, 203, 102, 103, 65, 81, 105,
75, 82, 95, 96, 97, 152, 153]
```

The sizes of the levels should be kept under the level multiplier ratio. And the compaction task:

```
Upper L1 [224.sst 7cd080e..=33d79d04]
Lower L2 [210.sst 1c657df4..=31a00e1b, 211.sst 31a00e1c..=46da9e43] -> [228.sst
7cd080e..=1cd18f74, 229.sst 1cd18f75..=31d616db, 230.sst 31d616dc..=46da9e43]
```

...should only have one SST from the upper layer.

Note: we do not provide fine-grained unit tests for this part. You can run the compaction simulator and compare with the output of the reference solution to see if your implementation is correct.

Task 2: Integrate with the Read Path

In this task, you will need to modify:

```
src/compact.rs
src/lsm_storage.rs
```

The implementation should be similar to simple leveled compaction. Remember to change both get/scan read path and the compaction iterators.

Related Readings

Leveled Compaction - RocksDB Wiki

Test Your Understanding

- What is the estimated write amplification of leveled compaction?
- What is the estimated read amplification of leveled compaction?
- Finding a good key split point for compaction may potentially reduce the write amplification, or it does not matter at all? (Consider that case that the user write keys beginning with some prefixes, `00` and `01`. The number of keys under these two prefixes are different and their write patterns are different. If we can always split `00` and `01` into different SSTs...)
- Imagine that a user was using tiered (universal) compaction before and wants to migrate to leveled compaction. What might be the challenges of this migration? And how to do the migration?
- And if we do it reversely, what if the user wants to migrate from leveled compaction to tiered compaction?
- What happens if compaction speed cannot keep up with the SST flushes for leveled compaction?

- What might needs to be considered if the system schedules multiple compaction tasks in parallel?
- What is the peak storage usage for leveled compaction? Compared with universal compaction?
- Is it true that with a lower `level_size_multiplier`, you can always get a lower write amplification?
- What needs to be done if a user not using compaction at all decides to migrate to leveled compaction?
- Some people propose to do intra-L0 compaction (compact L0 tables and still put them in L0) before pushing them to lower layers. What might be the benefits of doing so? (Might be related: PebblesDB SOSP'17)
- Consider the case that the upper level has two tables of `[100, 200], [201, 300]` and the lower level has `[50, 150], [151, 250], [251, 350]`. In this case, do you still want to compact one file in the upper level at a time? Why?

We do not provide reference answers to the questions, and feel free to discuss about them in the Discord community.

Bonus Tasks

- **SST Ingestion.** A common optimization of data migration / batch import in LSM trees is to ask the upstream to generate SST files of their data, and directly place these files in the LSM state without going through the write path.
- **SST Selection.** Instead of selecting the oldest SST, you may think of other heuristics to choose the SST to compact.